Graphic Era deemed to be University
DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

ForceForge: An Interactive 2D Newtonian Physics Simulation Sandbox

Team Name: *Team #*	*Team Jkd* *DSCPP-III-2026-T098*
Team member 1 (Team Lead)	*Negi, Pranjal – 2510370602* *2510370602@geu.ac.in* 
Team member 2	*Rawat, Dilasha – 2510012530* *2510012530@geu.ac.in* 
Team member 3	*Jakhmola, Rishabh – 25101210555* *25101210555@geu.ac.in* 
Team member 4	*Chhabra, Akshit – 2510011905* *2510011905@geu.ac.in* 

PHASE-III PROJECT PROGRESS DESCRIPTION

Project Abstract

ForceForge is an open, educational 2D Newtonian physics simulation sandbox built with C++17 and SFML. In textbooks and classrooms, some of the most interesting topics in mechanics are taught only through equations and static diagrams. ForceForge lets students watch gravity, momentum, collisions and projectile motion unfold in real time, change the parameters themselves, and see what happens.
Users can fire projectiles from a launcher, click to spawn circles, triangles and squares, and set the mass, speed and launch angle of each body. Any body can be selected and edited at runtime, which helps students explore how each parameter affects the motion.
ForceForge includes four presets that demonstrate core physics concepts: projectile motion, collisions, [third preset], and free fall. In the free-fall demo, three bodies of different mass are released together and land at the same time, showing that gravitational acceleration does not depend on mass.
ForceForge also demonstrates object-oriented design in C++. It uses STL containers to manage bodies, [spatial partitioning / broad-phase method] for collision detection, and feature-based collision response. Together, these let students learn physics by seeing it in action.

Updated Project Approach and Architecture

ForceForge is a desktop program built with C++17. SFML handles the window drawing and input. The same code works with SFML 2.6. SFML 3 is supported by using a compile-time version switch. Other than SFML only the C++ standard library is used. There is no network part; no communication protocol is needed. The design has four layers (see figure on the page). The interface turns mouse and keyboard input into events. Scene changes wait in a FIFO event queue. They are applied at one fixed point in each frame so that the scene never changes during a physics step or during a draw. The World class manages the scene, which includes bodies, springs and physics settings and moves it in fixed steps of 1/60 second: forces (gravity, drag and springs via the ForceGenerator class) – Euler integration – then collisions. The Collision System uses a hash grid to find candidate pairs, shape tests (the Separating Axis Theorem for polygons) to find contacts and impulses for bounce and Coulomb friction followed by position correction. Scene history stores two stacks of scene snapshots for undo and redo. Phase III additions. Drag and drop: a press on a body becomes a drag the mouse moves four pixels, a click still only selects. The grab offset is stored so that the body does not jump to the cursor and World::dragBody holds the body still and keeps it inside the walls and, above the floor. A Begin Drag event saves a point first. Friction control: the [. Keys adjust the friction coefficient, between zero and one point five. Each change is sent as a ReplaceScene event. It goes through the queue as every edit and can be undone.

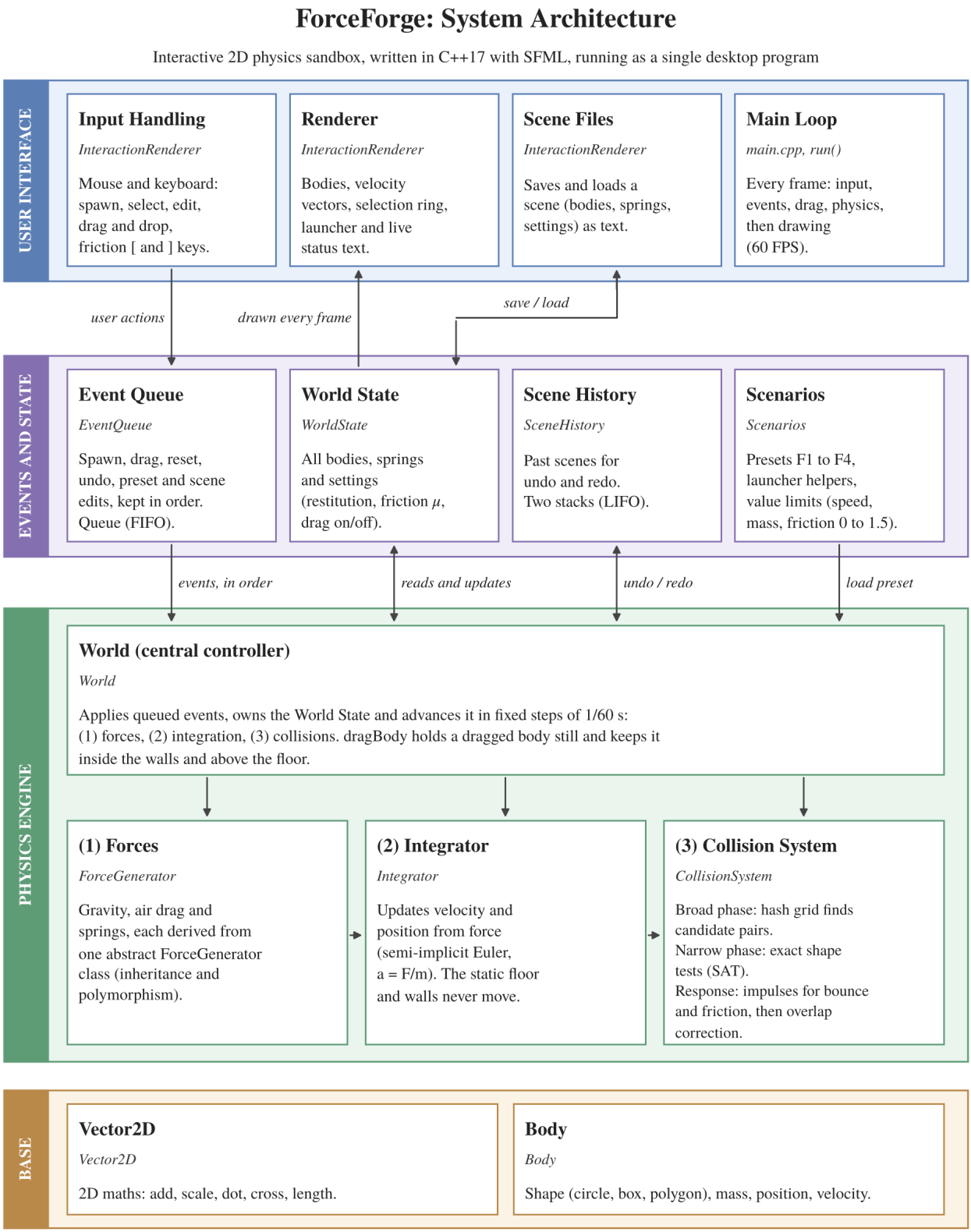


Figure: ForceForge system architecture (final, Phase III)

Tasks Completed

Task Completed	Team Member
Narrow-phase collision tests (circle, box, SAT polygons), spatial hash grid broad phase, CollisionSystegm::step pipeline, presets F1–F4 and launcher helpers. **Phase III:** live friction control with the [and] keys and its 0 to 1.5 limits.	Pranjal Negi
Gravity,drag and spring force through the Forcegenerator class,semi-implicit Euler integrator,impulse response witgh restitution and friction, undo and redo stacks,queue and event queue.**Phase III:** dragged bodies are kept inside the walls and above the floor.	Dilasha Rawat
SFML window,rendering and input,launcher interface,selectingwith on-screen values, scene saveand load, main.c++,support for SFML 2.6 and 3. **Phase III:** mouse-move input that completes drag aggnd drop, selection ring, updated controls list.	Rishabh Jakhmola
Vector2D maths, Body data type for all shapes, World class with fixed time step and body picking.	Akshit Chhabra
Automated test suite(123 checks,all passing), README,integration of all modules and final demonstration	Whole team

Challenges/Roadblocks

- Drag never started. Phase II had drag functions. The program never read mouse movement. We added a MouseMove input for both SFML versions; a press turns into a drag after four pixels of movement so clicks and drags stay separate. Drag never started. Phase II had drag functions. The program never read mouse movement. We added a MouseMove input for both SFML versions; a press turns into a drag after four pixels of movement so clicks and drags stay separate.
- Dragging into walls. A body pushed into the floor. A wall was shot out by the solver when released. World::dragBody now clamps the position using the body's bounding radius. Dragging into walls. A body pushed into the floor. A wall was shot out by the solver when released. World::dragBody now clamps the position using the body's bounding radius..
- Wrong jumping. The old loop used draw window pixels. Input is now converted to world coordinates. The grab offset is kept so the body does not jump to the cursor. Wrong jumping. The old loop used draw window pixels. Input is now converted to world coordinates. The grab offset is kept so the body does not jump to the cursor.
- Undo for actions. A BeginDrag event saves the scene before a drag and friction changes go through the event queue so Ctrl+Z restores both. Undo for actions. A BeginDrag event saves the scene before a drag and friction changes go through the event queue so Ctrl+Z restores both.
- Floating-point values. Repeated 0.05 steps gave values such as 0.30000001; each new value is rounded to two decimals. Clamped to 0 to 1.5. Floating-point values. Repeated 0.05 steps gave values such, as 0.30000001; each new value is rounded to two decimals. Clamped to 0 to 1.5.
- Presets set their friction. F2 and F3 use μ=0 to isolate momentum so the status line always shows the value and users adjust it after loading a preset. Presets set their friction. F2 and F3 use μ=0 to isolate momentum so the status line always shows the value and users adjust it after loading a preset.
- Keeping everything working. Every change was checked against the 123-checksuite before merging. Keeping everything working. Every change was checked against the 123-checksuite before merging.

Project Outcome/Deliverables

1. A working physics simulator is ready which is written in c++ 17 using SFML.A student can perform operations like spawning,launching,selecting,editing and dragging bodies in real time.The simulator lets a student to observe physics in real time.
2. The physics simulator contains four physics concepts that are projectile motion, elastic collision, inelastic collision and free fall,The physics simulator shows you what a textbook can make you think.
3. The full source code is provided which is easy to read and edit.There are 11 header file and 10 source file which contains an object model,force generator,event queue,undo redo stack and a spatial hash.
4. 123 Automated checks are included which cover maths.,collision,force,accuracy,etc.
5. Documentation and live demonstration is ready to be shown. The README provide prerequisites,build and run steps. The documentation also include Phase I to Phase III reports.

Progress Overview

ForceForge is complete and ready for the final demonstration. The object model, simulation loop, forces, integration, collision detection and response, presets, interface, undo/redo and tests all work together, and the full test suite passes.

In Phase III the team completed **drag and drop**, which was only partly in place in Phase II, and added **live friction control**. These were the last interactive features of the plan: students can now rearrange a scene by hand and change the surface from ice to rough while the simulation runs.

Ahead of the original plan: undo/redo with stacks, the editable launcher with body selection, compatibility with both SFML 2.6 and 3, and the automated test suite were not part of the original proposal.

Codebase Information

Repository: https://github.com/Pranjal-ext/ForgeForce **Branch:** main

Structure: include/ (11 headers), src/ (10 source files including main.cpp), tests/test_main.cpp, README.md, .gitignore. About 1,900 lines of program code and 670 lines of tests, not counting blank lines and comments.

Build: C++17 (g++ 9 or newer) linked against SFML 2.6 or 3. The test program builds from the same sources with tests/test_main.cpp in place of src/main.cpp.

Important Phase III changes:

- InteractionRenderer.hpp / .cpp: MouseMove input, drag start after 4 px, grab offset, drop on release, friction keys [and], selection ring, updated controls list.
- World.cpp: dragBody keeps a dragged body inside the walls and above the floor.
- Scenarios.hpp: MIN_FRICTION (0.0) and MAX_FRICTION (1.5) limits.

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
Total: 123 automated checks, 123 passed, 0 failed.		
Unit: Vector2D and Body	Pass	13 checks: vector operations,mass and inverse mass static body, polygon vertex and point test.
Unit: Collision detection	Pass	12 checks: circle,box,triangle and hexagon pairs(SAT). Near misses,direction no NaN for identical positions
Unit:Collision response	Pass	3 checks:mass elastic collision swaps velocities momentum conserved static bodies never move
Unit:Forces	Pass	12 checks:F equals m g drag opposes motion spring F equals k x with opposite ends dispatch through the base class
Unit:Integration	Pass	5 checks:v equals a t forces cleared each step,semi-implicit Euler keeps spring energy bounded,Euler gains energy.
Accuracy: Projectile range	Pass	Range versus v squared sine 2 theta over g: 1.7 percent error at 60 steps per 0.4 percent at 240 steps per second.
Accuracy: Spring period	Pass	Measured period within 2 percent of 2 pi root m over k.
Integration: World and simulation behaviour	Pass	23 checks: fixed-step timing all shapes land and rest on the floor bounce then settle, stack, friction stops sliding walls contain fast bodies.
Performance: 100 bodies	Pass	No NaN no escapes physics step under the 8 milliseconds limit. [Add milliseconds per step measured on your machine]
Unit: Event queue and undo/redo stacks	Pass	16 checks: FIFO order, LIFO undo, capacity limit redo cleared by an action reset can be undone.
Functional: Presets	Pass	11 checks: all four load; elastic and inelastic results match theory; inelastic momentum conserved (1200); free-fall bodies of mass fall together.
Functional: Launcher and scene files	Pass	25 checks: angle, speed and mass limits; 20 shots at speed stay, in the arena; save load round trip; damaged or missing files rejected.

Test Type	Status (Pass/Fail)	Notes
Functional: Drag and drop (Phase III)	Pass	7 scripted checks: click vs drag, body follows mouse with grab offset, one undo point, clamped inside arena, drop on release, Ctrl+Z restores.
Functional: Friction control (Phase III)	Pass	6 scripted checks: [and] steps (Shift for 0.01), value reaches the collision system, limits 0 and 1.5, Ctrl+Z restores.
Manual: interactive run in SFML window	*Pass*	*All checks were passed.* .

Deliverables Progress

Deliverable	Status	Remarks
C++ source code and object models	Completed	Body data type with Shapetype (circle,polygon,box)
Simulation and integration engine	Completed	Fixed one by sixtys,semi-implicit euler.
Force generation (gravity,spring,drag)	Completed	Polymorphism in ForceGen.
Collision system with impulse response	Completed	Bounce,coulomb friction,position corrections,sleep.
Spatial partitioning	Completed	Spatial hashgrid in broad phase.
SFML Interface and rendering	Completed	Launcher,selection,drag and Drop,live friction control
Preset demonstrating	Completed	Four presets,F1 to F4
Scene save or loadd	Completed	Functions implemented and tested
Automated test suite	Completed	123/123 checks pass.
Documentation and final demonstration	Completed	README and Phase I to III reports are ready